OPEN SOURCE SECURITY MONITORING FOR HOSPITALS

# Healthcare Threat Monitor

Technical reference: how every part works

| | |
|---|---|
| Version | 0.9.0 |
| Date | November 2026 |
| Licence | Apache License 2.0 |
| For | Hospital IT staff, security reviewers, auditors and developers |
| Project page | sea4real2000.github.io/Healthcare-threat-monitor |
| Source code | github.com/sea4real2000/Healthcare-threat-monitor |

> This PDF is made from docs/technical_reference.md in the repository, which is always the latest version.

This document explains how every part of the platform works: what runs where, how a log becomes an incident, every API with examples, every setting, how the platform protects itself, and how to run it day to day. It is written for hospital IT staff, security reviewers, auditors and developers. You do not need to be a programmer to follow most of it; where code appears, the text around it says what it does. Words that may be unfamiliar are explained in the glossary at the end.

Other documents cover narrower ground and are linked where they matter:

| Document | What it covers |
|---|---|
| hospital_install.md | Installing in a hospital, step by step, and what has and has not been tested |
| detection.md | The detection method in more depth |
| threat_model.md | How the platform itself could be attacked, and what limits each attack |
| public_demo.md | Running the public demonstration on free hosting |
| README.md | Starting points for developers |
| CHANGELOG.md | What changed in each version |

## Contents


1. The platform in one page
2. How the parts fit together
3. Following one log through the platform
4. Collecting logs
5. Patient record access
6. Detection
7. Response and forwarding
8. The API
9. What is stored
10. Configuration
11. Accounts and sign in
12. How the platform protects itself
13. Backups, restores and updates
14. Running it day to day
15. How it is tested
16. Standards and outside services used
17. Glossary
18. Appendix: every setting


## 1. The platform in one page

Healthcare Threat Monitor watches the security logs a hospital already produces and warns early about three kinds of harm: ransomware and other attacks spreading through the network, staff opening patient records they have no reason to open, and medical devices behaving in ways they never should. A person reviews every warning and approves every response.

It works in seven steps:

| Step | What happens |
|---|---|
| Collect | Logs arrive from Windows computers, routers, firewalls, servers, the patient record system and other sources. Each record is turned into one common format, called an event. |
| Detect | Twenty two written rules catch known signs of attack. Three learning models flag activity that is unusual for this hospital. A rule or model that finds something raises an alert. |

| Step | What happens |
|---|---|
| Correlate | Alerts about the same machine or person within a short time join one incident, with one confidence score and a band: low, medium or high. |
| Review | An analyst sees the evidence, the attacker techniques involved and the reasons behind each alert, then confirms or dismisses the incident. |
| Respond | For a high incident, an analyst can approve cutting a machine off the network. Incidents can also be sent to a hospital's central security system. |
| Record | Every alert, decision and action is written to an audit log that shows if anyone changes a past entry. |
| Learn | A dismissal teaches the detectors that raised it, so the same false alarm is not raised again. |

Everything runs on one computer inside the hospital. Nothing is sent to the project or to anyone else unless an administrator sets up forwarding to the hospital's own systems.

There are three ways to run it:

| Way | For | What it is |
|---|---|---|
| Appliance | Hospitals | A ready made virtual machine (`.ova` for VirtualBox and VMware, `.vhdx` for Hyper-V) or a USB installer for a spare computer. Set up with clicks. |
| Hospital installation with Docker | Hospitals that already run Docker | `docker-compose.hospital.yml` on a Linux server. The appliance runs exactly this inside. |
| Demonstration | Anyone trying it | `docker compose up`, or hosted on Vercel and Supabase. Runs a simulated hospital with test attacks. |

A hospital installation never runs the simulator, never accepts test attacks, has no public demo login, and needs sign in for every page.

## 2. How the parts fit together

### The parts

| Part | Language | Folder | What it does |
|---|---|---|---|
| Console | TypeScript, Next.js | `web` | The web pages staff use, and the API every other part talks to. |
| Database | PostgreSQL 16 | `supabase/migrations` | Holds events, alerts, incidents, settings, accounts and the audit log. |
| Background service | Python | `engine` | Detection, model training, reading records databases and FHIR servers, forwarding, retention. |

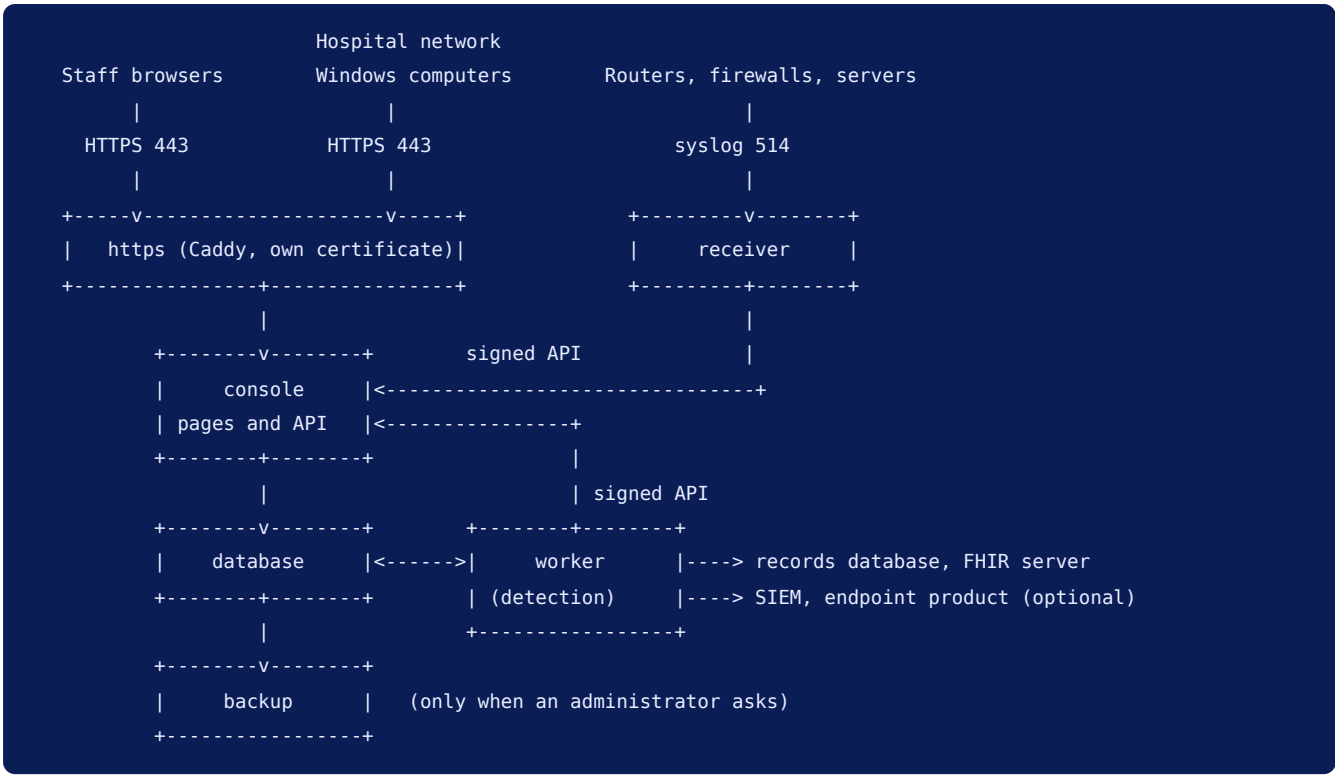
| Part | Language | Folder | What it does |
|---|---|---|---|
| Log receiver | Python | `engine/receiver` | Listens for syslog from routers, firewalls and servers on port 514. |
| Windows collector | PowerShell and C# | `web/collectors/windows` | Installed on Windows computers; sends their security events. |
| HTTPS server | Caddy | `docker/Caddyfile` | Serves the console over HTTPS with the installation's own certificate. |
| Backup service | Node.js | `web/scripts/backup_service.mjs` | Makes and restores backups when an administrator asks. |
| Appliance | Shell scripts | `appliance` | Builds the virtual machine and USB installer, and runs the screen. |

## The hospital installation

`docker-compose.hospital.yml` starts these services. Each runs in its own container.

| Service | Image | Role | Reachable from the network |
|---|---|---|---|
| `secrets` | tools | Makes the installation's secrets on the first start, then keeps them. Runs once and stops. | No |
| `db` | `postgres:16-alpine` | The database. | No |
| `setup` | tools | Applies database changes (migrations) and prepares the first visit. Runs once at each start and stops. | No |
| `console` | console | The web pages and the API. | Only through `https` |
| `https` | `caddy:2` | HTTPS in front of the console, on ports 80 and 443. | Yes: 80, 443 |
| `worker` | worker | The background service. | No |
| `receiver` | worker | The log receiver, on port 514 (UDP and TCP). | Yes: 514 |
| `backup` | backup | Backups and restores. | No |

The secrets live in a private Docker volume, readable only by these containers. The database port and the console's own port are never published, so the only ways in from the network are HTTPS and syslog.




### The appliance

The appliance is a small Ubuntu 24.04 system that carries the images above and starts them. It has no remote sign in of any kind. Four of its own services matter:

| Service | What it does |
|---|---|
| `htm-firstboot` | On the first start, grows the disk to its full size and loads the platform's software into Docker. Can be switched off at any point; it carries on next time. |
| `htm` (`htm-start`) | Starts the platform at the computer's current network address, so the certificate matches what staff type. |
| `htm-screen` | Shows the address, the setup code and the state on the computer's own screen. Press N for a fixed network address. |
| `htm-installer` | On the USB stick only: copies the platform onto a disk after the disk number and the word ERASE are typed. |

## 3. Following one log through the platform

This is what happens when a nurse's computer runs a command that deletes backup copies, a common step just before ransomware locks files.

1. **The computer records it.** Windows writes a process creation event (number 4688) to its Security log.
2. **The collector sends it.** Within a minute, the Windows collector's scheduled task reads new events, keeps only the fields the platform reads, signs the batch with its group's secret and sends it over HTTPS to `POST /api/ingest`. It trusts only this installation's own certificate authority.

3. **The console checks and stores it.** The console checks the signature, the time stamp and that the group is switched on, turns each record into an event in the common format, matches it to the computer in the inventory, and stores it. A batch sent twice is stored once.
4. **The background service reads it.** Every few seconds (`engine.poll_seconds`), the worker reads new events and passes each through every rule. The rule `ransom_shadow_copy_delete` matches, with confidence 0.85, and raises an alert naming the MITRE ATT&CK technique T1490.
5. **Alerts become an incident.** The alert joins any incident about the same computer or person in the last 45 minutes, or opens a new one. The incident's confidence combines its alerts.
6. **The band decides what happens.** With only one log source agreeing, the incident cannot pass 0.79, so it is medium: it goes to the analyst queue and, if set up, is forwarded to the hospital's SIEM. If the firewall then logs the same computer scanning the network, two sources agree and the incident can become high.
7. **A person decides.** The analyst opens the incident, sees each alert's evidence and reasons, and either dismisses it (the detectors learn) or confirms it. For a high incident, the analyst can approve isolating the computer.
8. **The response is carried out.** The approval waits as a request. The endpoint connector reads it, asks the endpoint product to isolate the computer, and reports back. Only then is the computer marked isolated.
9. **Everything is recorded.** The incident opening, the analyst's decision, the approval and the isolation each add an entry to the audit log.

## 4. Collecting logs

### The ten log sources

Every record is turned into an event with the same fields: when, which source, which machine, which person, what kind of action, its outcome and the details. These are the sources the platform reads:

| Source | Typical origin | Examples of what it shows |
|---|---|---|
| `windows` | Windows collector | Sign ins, failed sign ins, new accounts, programs started, services installed, the security log being cleared |
| `firewall` | Log receiver, or a collector | Allowed and blocked connections, with addresses and ports |
| `vpn` | Log receiver | Remote sign ins and where they come from |
| `linux` | Log receiver | Server sign ins and failures over SSH |
| `ehr` | Records database, FHIR server or upload | Who opened which patient's record, and when |
| `edr` | Endpoint detection product | Files written, renamed and deleted, programs started |
| `dns` | DNS server | Domains looked up |
| `email` | Email security product | Messages and attachments delivered or blocked |
| `medical_device` | Network monitoring | Connections, hosts and packet rates per device |

| Source | Typical origin | Examples of what it shows |
|---|---|---|
| `cloud` | Cloud audit logs | Actions in cloud services |

A source can be switched off on the Sources page. Records for a switched off source are refused with the reason `source_disabled`.

### The Windows collector

An administrator creates a group of Windows computers under **Sources**, **Windows computers**, and downloads one installer file for it. Run on a computer as an administrator, it:

- copies the collector to `C:\ProgramData\HealthcareThreatMonitor`, which only SYSTEM and administrators can open;
- switches on process auditing, so programs started are recorded;
- adds a scheduled task that runs every `windows.interval_minutes`, and when the computer starts, as the SYSTEM account.

Each run reads new events of these kinds and sends them:

| Log | Event | Meaning |
|---|---|---|
| Security | 4624 | Successful sign in |
| Security | 4625 | Failed sign in |
| Security | 4672 | Special privileges given at sign in |
| Security | 4688 | A program started |
| Security | 4698 | A scheduled task created |
| Security | 4720 | A user account created |
| Security | 4732 | A member added to a local security group |
| Security | 4740 | An account locked out |
| Security | 1102 | The security log cleared |
| System | 7036 | A service started or stopped |
| System | 7045 | A new service installed |

The position reached in each log is saved only after the platform confirms it stored the events, so nothing is lost when the network is down. On its very first run it sends only the last 24 hours.

The installer carries the installation's own certificate authority, and the collector trusts only that authority, so it cannot be tricked into sending events to another server. Each group has its own secret; switching a group off refuses its computers at once.

### The log receiver

Routers, firewalls and servers send their logs by syslog to port 514 of the installation, over UDP or TCP. Both common syslog layouts are read (RFC 3164 and RFC 5424). Each message is then matched against these formats:

| Format | Becomes |
|---|---|
| Cisco ASA | Firewall events |
| Fortinet FortiGate | Firewall and VPN events |
| pfSense and OPNsense filter log | Firewall events |
| Linux netfilter, including UFW and iptables | Firewall events |
| MikroTik | Firewall events |
| Linux sign in messages (sshd) | Linux events |
| Other key=value firewall logs | Firewall events, when the fields are recognisable |

A device that sends for the first time waits on the Sources page until an administrator accepts it. Nothing from it is kept before that. Messages longer than `receiver.max_message_bytes` are cut; more than `receiver.sender_per_minute` messages a minute from one device are counted and dropped. Messages in a format the platform does not read are counted, so the Sources page shows what each device sends.

The receiver works out whether each address is inside the hospital or on the internet, and which machine it is from the inventory, so the network rules apply to router logs too.

### The inventory

Detection needs to know the hospital: departments, staff and their roles, computers and devices, and patients with flags such as "staff member" or "high profile". The setup wizard under **Setup** loads these from spreadsheets, with a template for each ( `GET /api/templates/{kind}` ). Each row is checked, and a faulty file is refused with the row numbers to fix.

## 5. Patient record access

Watching who opens which patient record is one of the platform's main purposes. There are three ways to bring in the record system's access log. All three produce `ehr` events in the same format.

### A. A read only database connection (recommended)

An administrator connects the record system's database under **Sources**, **Patient records**. PostgreSQL, MySQL and MariaDB are supported.

1. The administrator gives the address, database name, and a database account that can only read.
2. The platform lists the tables, guesses which columns hold the time, the staff member, the patient and the action, and shows a preview of the rows it would read.

3. Once saved, the background service reads new rows every `sources.poll_seconds`, at most `sources.batch_rows` at a time, and remembers where it stopped.

The connection refuses to write anything: every session is opened read only. The database password is sealed (encrypted) with `SOURCES_SECRET` before it is stored, and only the background service unseals it, when it connects.

### B. A FHIR server

Many record systems offer FHIR, the international standard for exchanging health data. Its `AuditEvent` resource records who opened which record. The platform asks the server for new AuditEvents every poll. Four ways of signing in to the server are offered:

| Way | When to use it |
|---|---|
| None | A test server only |
| Token | The vendor issued a fixed access token |
| Client secret | OAuth client credentials: a client ID and secret |
| Private key | SMART Backend Services. The console makes a key pair; the vendor registers its public half. Each request for an access token is signed with the private half (RS384), with the scope `system/AuditEvent.read` unless another is set. |

Records are only ever read from the server's own address; a "next page" link pointing anywhere else is refused. Access tokens come from the token address the administrator gives, or the one the server publishes.

### C. An uploaded report

Where neither is possible, an administrator uploads an access report exported from the record system as CSV or a spreadsheet. Unreadable rows are listed with the reason; rows uploaded twice are stored once.

## 6. Detection

This section is a summary. detection.md explains the method in more depth.

### Rules

Rules are rows in the `rules` table and can be changed under **Rules** without touching code. Each has a kind:

- **match**: one event meets every condition;
- **threshold**: enough matching events for one machine or person within a time window;
- **new value**: a machine or person does something it has never done before.

The rules shipped with 0.9.0:

| Area | Rule | Kind | ATT&CK | Confidence |
|---|---|---|---|---|
| Ransomware | Shadow copies or backup catalogue deleted | match | T1490 | 0.85 |
| Ransomware | Credential dumping command | match | T1003.001 | 0.80 |
| Ransomware | Mass file rename on one machine | threshold | T1486 | 0.80 |
| Ransomware | Ransom note file created | match | T1486 | 0.75 |
| Ransomware | Windows security log cleared | match | T1070.001 | 0.70 |
| Ransomware | Office application started a script interpreter | match | T1204.002 | 0.60 |
| Ransomware | Backup or shadow copy service stopped | match | T1489 | 0.50 |
| Ransomware | Message judged malicious was still delivered | match | T1566 | 0.50 |
| Ransomware | Risky attachment type reached an inbox | match | T1566.001 | 0.35 |
| Access | Activity from a terminated account | match | T1078 | 0.80 |
| Access | One machine signing in to many others | threshold | T1021 | 0.60 |
| Access | Special privileges for a non administrator | match | T1078 | 0.55 |
| Access | Repeated failed sign ins for one account | threshold | T1110 | 0.50 |
| Access | Remote access from a new country | new value | T1078 | 0.35 |
| Records | Flagged patient record opened from another department | match | T1213 | 0.45 |
| Network | Lookup of a known malicious domain | match | T1071 | 0.80 |
| Network | Internal port scan | threshold | T1046 | 0.60 |
| Network | Large upload to the internet | match | T1048 | 0.60 |
| Network | Lookup of a domain never seen before | new value | T1071 | 0.25 |
| Devices | Medical device sending a flood of packets | match | T1498 | 0.60 |
| Devices | Medical device reaching for the internet | match | T1071 | 0.60 |
| Devices | Medical device contacted an unfamiliar host | new value | T1046 | 0.60 |

## Learning models

Three models each watch one kind of thing over short windows of time: staff record access (15 minutes), machine behaviour (1 minute) and medical device traffic (5 minutes). Each is an Isolation Forest, a method that learns what normal looks like without being shown attacks. A window raises an alert only when it is unusual overall and at least one measure is far outside the normal range for its peer group. Every alert lists the measures that drove it, with the value seen and the usual value. The models retrain every `engine.retrain_hours` on the last `engine.training_hours`.

### Confidence and bands

Alerts from different detectors are treated as independent witnesses: if one is 60% sure and another 50%, the chance at least one is right is 1 minus (0.4 times 0.5), which is 80%. The incident's band follows from its confidence:

| Band | Confidence | What happens |
|---|---|---|
| Low | below `detection.band_medium` (0.5) | Kept for tuning. Not queued, not forwarded. |
| Medium | from 0.5 | Goes to the analyst queue and is forwarded to the SIEM. |
| High | from `detection.band_high` (0.8) | As medium, and an analyst may approve isolation. |

**An incident cannot be high unless at least two independent log sources agree** (`detection.high_min_sources`). Until then it is capped at 0.79. A single faulty or faked sensor can therefore never lead to an isolation offer by itself.

### The learning period

For the first `learning.days` (7) after each kind of log starts arriving, the platform learns what is normal in this hospital: which addresses, domains and countries are usual, and how much each person and machine normally does. During that time, clear signs of attack are still reported straight away; activity that is only unusual is reported once learning ends. **Sources** shows each kind of log's learning, with **Finish now** and **Learn again**.

### Learning from analysts

When an analyst dismisses an incident as expected, each detector that contributed learns for that machine or person only: a new value joins its normal set, a threshold rises by `feedback.threshold_factor`, a match rule is silenced, and a model needs a larger difference before it alerts again. Every such change is listed on the Rules page and can be undone. A confirmed incident's activity is kept out of future training.

## 7. Response and forwarding

### Isolation, with a person's approval

1. An analyst opens a high incident and approves isolation of a machine it names. Isolation is only offered on a high incident that names that machine and has not been dismissed. On the demonstration, the public demo visitor can also approve, within a rate limit.
2. The request waits in `response_actions` with the status `requested`.
3. A connector, a small program with the respond permission, reads waiting requests (`GET /api/response/actions`), acts on the endpoint product, and reports the result (`POST /api/response/actions/{id}/ack`) as `acknowledged` or `failed`.
4. Only an acknowledgement changes the machine's status to isolated. A failure leaves it unchanged and is shown on the incident.
5. Releasing a machine works the same way.

Out of the box, a hospital installation has no connector: approvals wait until one is set up. `engine/connectors/webhook_connector.py` posts each approved request to a webhook in the hospital's endpoint product, signed with `CONNECTOR_TARGET_KEY` and with an Authorization header from `CONNECTOR_TARGET_AUTH`. For a product with its own API, copy that file and replace its `deliver()` function. In the demonstration, the background service plays the endpoint itself (`response.simulated_connector`).

### Forwarding to a SIEM

When an incident reaches medium or high, rises in band, is decided by an analyst, or has a machine isolated or released, a message is queued in `siem_outbox` and sent in up to two formats:

- **A JSON webhook** to `siem.webhook_url`, signed with HMAC SHA256 using `SIEM_WEBHOOK_KEY`, with the headers `x-htm-timestamp` and `x-htm-signature` (the same signing scheme as the API, below). Empty sends it to the built in receiver, so the forwarding can be watched working.
- **A CEF line** inside an RFC 5424 syslog frame, only when `siem.syslog_host` is set, to that host on `siem.syslog_port` over `siem.syslog_protocol`. Severity is 9 for high, 6 for medium and 3 for low.

A message that cannot be delivered is retried up to `siem.max_attempts` times, so nothing is lost when a destination is briefly down. Each message carries the newest audit log entry's hash at the time of sending. A copy kept outside the platform makes it possible to notice if the end of the audit log is ever cut off.

## 8. The API

The console's API is how every other part talks to it. Answers are JSON, except file downloads (the evaluation report, spreadsheet templates, the certificate, backups and pictures). A refusal from a JSON endpoint has this shape, with HTTP status codes used as usual (400 bad request, 401 not signed in or wrong signature, 403 not allowed, 404 not found, 409 conflict, 413 too large, 429 too many requests, 500 internal error, 503 not available):

```
{ "ok": false, "error": "signature_mismatch" }
```

### Four ways a request proves who it is from

| Way | Used by | How |
|---|---|---|
| Collector signature | Collectors, the receiver, the background service, connectors | HMAC SHA256 signature in three headers, below |
| Signed in session | Staff in a browser | The `htm_session` cookie, set at sign in. Requests that change something must also come from the installation's own pages. |
| Setup code | A new installation nobody has set up yet | The `X-Setup-Code` header, for restoring a backup only |

| Way | Used by | How |
|---|---|---|
| SIEM key | The built in SIEM receiver | HMAC SHA256 with `SIEM_WEBHOOK_KEY` |

### Signing a request as a collector

Every collector has an id, such as `win_ab12cd34ef` or `hospital_service`, and a secret. The secret is never stored: it is worked out from the installation's master key each time, as HMAC SHA256 of `collector:{id}` with `INGEST_SIGNING_KEY`. Only collectors registered and switched on in the `collectors` table are accepted.

To sign a request, build this text, where the body hash is the SHA256 of the exact body sent, in lower case hexadecimal (an empty body for a GET):

```
v1:{timestamp}:{METHOD}:{path}:{sha256 of the body}
```

then send three headers:

| Header | Value |
|---|---|
| `x-htm-key-id` | The collector id |
| `x-htm-timestamp` | The time now, in Unix seconds |
| `x-htm-signature` | HMAC SHA256 of the text above with the collector's secret, in hexadecimal |

A request whose time stamp is more than `ingest.max_skew_seconds` (300) from the server's clock is refused, so a captured request cannot be replayed later. Requests are also limited per address by `limits.api_per_minute`.

A complete example in Python:

```
import hashlib, hmac, json, time, uuid, urllib.request

API = "https://10.0.0.5"
COLLECTOR_ID = "my_collector"
SECRET = "the secret printed for this collector"

def send(method, path, body=b""):
    timestamp = str(int(time.time()))
    text = f"v1:{timestamp}:{method}:{path}:{hashlib.sha256(body).hexdigest()}"
    signature = hmac.new(SECRET.encode(), text.encode(), hashlib.sha256).hexdigest()
    request = urllib.request.Request(API + path, data=body or None, method=method, headers
={
        "content-type": "application/json",
        "x-htm-key-id": COLLECTOR_ID, "x-htm-timestamp": timestamp, "x-htm-signature": sig
nature})
    return json.load(urllib.request.urlopen(request))

batch = {"batch_id": str(uuid.uuid4()), "records": [
    {"source": "firewall", "payload": {"id": "fw-000123", "ts": "2026-11-02T10:15:06Z", "a
ction": "deny",
     "src": "203.0.113.45", "dst": "10.20.0.1", "dpt": 22, "proto": "tcp"}}]}
print(send("POST", "/api/ingest", json.dumps(batch).encode()))
```

`engine/simulator/client.py` is a complete reference client, used by the background service itself. To print a collector's secret: `INGEST_SIGNING_KEY=... npm run --silent collector:key -- my_collector` in the `web` folder.

## Endpoints for collectors and connectors

| Method and path | Purpose |
|---|---|
| `POST /api/ingest` | Send a batch of log records |
| `POST /api/inventory` | Register or update departments, staff, machines and patients |
| `GET /api/runs/{run_id}` | What the database holds for one simulator run, per source (testing) |
| `GET /api/response/actions` | Isolation and release requests waiting to be carried out. Needs the respond permission. |
| `POST /api/response/actions/{id}/ack` | Report that a request was carried out, or failed. Needs the respond permission. |

**`POST /api/ingest`** takes:

```
{
  "batch_id": "a new random UUID for each batch",
  "run_id": "optional, letters, digits and underscores",
  "records": [ { "source": "windows", "payload": { "...": "the record as the source wrote
it" } } ]
}
```

and answers:

```
{ "ok": true, "batch_id": "...", "replayed": false, "records": 2, "accepted": 2, "duplicat
e": 0, "rejected": 0, "rejects": [] }
```

A batch with the same `batch_id` sent again changes nothing and answers `"replayed": true`, so a collector can safely send again after a network failure. Each rejected record is listed with its position and reason (up to 20). Limits: `ingest.max_batch_records` records and `ingest.max_body_bytes` bytes per batch. A collector can be limited to certain sources ( `collectors.allowed_sources` ); a Windows group's collectors can only send `windows` records.

**`GET /api/response/actions`** answers with up to 100 waiting requests:

```
{ "ok": true, "actions": [ { "id": "42", "action": "isolate", "asset_id": "WS-ED-01", "ass
et_ip": "10.20.3.14",
  "asset_kind": "workstation", "incident_id": "117", "requested_by_name": "Ada Obi", "requ
ested_at": "..." } ] }
```

**`POST /api/response/actions/{id}/ack`** takes `{ "status": "acknowledged" }` or `{ "status": "failed", "note": "why, up to 200 characters" }`. A request that is no longer waiting answers 409 `action_not_waiting`.

## Endpoints for signed in staff

| Method and path | Who | Purpose |
|---|---|---|
| `GET /api/trust/root` | Any signed in account | Download the installation's certificate authority, `htm-root.crt` |
| `GET /api/collectors/windows/{id}` | Administrators | Download the installer for one group of Windows computers. It carries that group's secret. |
| `GET /api/audit/export` | Any signed in account | The whole audit log as JSON, for an outside auditor. Signed with `AUDIT_EXPORT_KEY` when set. `scripts/verify_audit_export.py` checks it independently. |
| `GET /api/evaluation/report` | Anyone | The latest detection evaluation report. It holds only results from the simulated hospital; a hospital installation, which runs no evaluation, has none. |
| `GET /api/templates/{kind}` | Anyone | A spreadsheet template for the setup wizard |
| `POST /api/backups` | Administrators | Ask for a backup: `{ "passphrase": "..." }`. Answers `{ "id": "..." }`. |
| `GET /api/backups/{id}/status` | Whoever holds the job id | How a backup or restore is going: `working`, `done` or `failed`, with the step |
| `GET /api/backups/{id}/file` | Administrators | Download a finished backup |

| Method and path | Who | Purpose |
|---|---|---|
| `POST /api/backups/upload` | Administrators, or the setup code on a new installation | Upload a backup file to restore, as the request body |
| `POST /api/backups/restore` | Administrators, or the setup code on a new installation | Restore an uploaded backup: `{ "upload": "...", "passphrase": "...", "confirm": "RESTORE" }` |

The backup endpoints refuse a request whose `Origin` header is not the installation itself, so another website cannot trigger them through a signed in browser.

### Other endpoints

| Method and path | Purpose |
|---|---|
| `GET /api/health` | Health check for monitors. Says whether the database answers and which migrations are applied. |
| `GET /api/health?diagnose=1` | A fuller report, with the header `x-htm-diagnose-token` set to `DIAGNOSE_TOKEN`: table counts, tables without row level security, and whether `DATABASE_URL` and `INGEST_SIGNING_KEY` are set. It never shows a secret. Off unless `DIAGNOSE_TOKEN` is set. |
| `POST /api/siem/receive` | The built in SIEM receiver, signed with `SIEM_WEBHOOK_KEY` |
| `GET /api/media/{slot}` | A landing page picture (demonstration) |

## 9. What is stored

Everything is in one PostgreSQL database. The main tables, grouped by purpose:

| Purpose | Tables |
|---|---|
| The hospital | `departments`, `staff`, `assets` (computers and devices), `patients` |
| Logs | `events`, `ingest_batches`, `ingest_rejects`, `sources`, `collectors`, `log_senders` |
| Records connections | `record_sources`, `record_uploads`, `inventory_imports` |
| Detection | `rules`, `models`, `baseline_values`, `detector_exceptions`, `model_training_labels`, `learning_periods` |
| Incidents | `alerts`, `incidents`, `incident_entities`, `feedback` |
| Response | `response_actions`, `siem_outbox`, `siem_inbox` |
| Accounts | `users`, `sessions`, `rate_limits` |
| Audit | `audit_log` |

| Purpose | Tables |
|---|---|
| Configuration | `settings`, `copy` (every label and sentence), `site_images`, `compliance_controls` |
| Running state | `engine_state`, `evaluation_runs`, `scenario_runs`, `schema_migrations` |

**Retention.** Events older than `retention.events_hours` (36) are removed. The evidence behind each alert is kept with the alert, so incidents keep their evidence after the events are gone. Alerts, incidents and the audit log are kept.

**Database changes.** Each file in `supabase/migrations` is safe to run again, and all are applied, in order, at every start of a hospital installation. Updating to a new version therefore needs no separate database step.

**Lockdown.** On a hosted demonstration with Supabase, the public database roles can read and write nothing; `supabase/tests/lockdown.sql` proves it, and CI checks that the test fails when a hole is opened.

## 10. Configuration

There are three layers, from rarely changed to often changed.

### Secrets (environment variables)

On a hospital installation and the appliance, the database password, `INGEST_SIGNING_KEY`, `AUTH_SECRET`, `SIEM_WEBHOOK_KEY`, `AUDIT_EXPORT_KEY` and `SOURCES_SECRET` are made automatically on the first start, as 64 random characters each, and kept in the private secrets volume. Nobody needs to type them. The optional `DIAGNOSE_TOKEN` (at least 24 characters) and `RESEND_API_KEY` are only set by hand. On a hosted demonstration every value is set in the hosting provider, and each secret must be at least 32 characters.

| Variable | Used by | Purpose |
|---|---|---|
| `DATABASE_URL` | Console, worker | Where the database is |
| `DATABASE_SSL` | Console | `require` or `disable`. Empty requires TLS except for local databases. |
| `INGEST_SIGNING_KEY` | Console | Every collector secret is worked out from it |
| `AUTH_SECRET` | Console | Seals each account's authenticator secret. Changing it means every account sets up its authenticator again. |
| `SIEM_WEBHOOK_KEY` | Console, worker | Signs forwarded incidents |

| Variable | Used by | Purpose |
|---|---|---|
| `SOURCES_SECRET` | Console, worker | Seals records database passwords and FHIR secrets and keys |
| `AUDIT_EXPORT_KEY` | Console | Signs audit log exports |
| `DIAGNOSE_TOKEN` | Console | Opens the detailed health report. At least 24 characters. |
| `RESEND_API_KEY` | Console | Sends invitation emails through Resend (optional) |
| `HTM_TRUST_URL` | Console | Where to read the installation's certificate authority for Windows installers |
| `HTM_BACKUP_DIR` | Console | The folder shared with the backup service |
| `HTM_API_URL` | Worker, connectors | The console's address |
| `HTM_COLLECTOR_ID`, `HTM_COLLECTOR_SECRET` | Worker | The collector the background service signs as |
| `HTM_CONNECTOR_ID`, `HTM_CONNECTOR_SECRET`, `CONNECTOR_TARGET_URL`, `CONNECTOR_TARGET_KEY`, `CONNECTOR_TARGET_AUTH` | Webhook connector | Carrying out isolations through a webhook |
| `HTM_DEVICE_DATASET` | Worker | Optional folder of CICIoMT2024 files for the demonstration |
| `HTM_HOSTNAME` | Hospital installation | The address staff type. The appliance sets it to its own address. |

## Settings

Every number the platform uses, such as thresholds, limits, intervals and bands, is a row in the `settings` table and can be changed by an administrator under **Settings**, with no restart. In the console, a missing or invalid setting is an error on purpose, so nothing silently falls back to a hidden default. The log receiver and forwarding keep safe built in values for a few settings, so logs are not lost if the database cannot be read for a moment. The appendix lists all 79 with what each does.

## Wording, rules and pictures

| To change | Where |
|---|---|
| Any label or sentence the console shows | **Settings**, **Labels and copy** |
| A detection rule, or add one | **Rules** |
| Landing page photographs | **Settings**, **Images** |
| The compliance mapping | Rows in `compliance_controls` |

| To change | Where |
|---|---|
| Colours, type and spacing | The token block at the top of `web/app/globals.css` |

## 11. Accounts and sign in

| Topic | How it works |
|---|---|
| Roles | **Administrator**: everything, including settings, accounts, sources and backups. **Analyst**: reads everything, reviews incidents, approves responses. |
| First administrator | On a new installation, every page leads to the first visit page, which needs the ten character setup code from the installation's own screen. The page closes for good once an administrator exists. |
| Other accounts | An administrator invites them under **Settings**, **Accounts**. The invitation link works for `auth.setup_link_hours`. |
| Passwords | At least `auth.password_min_length` (12) characters, not containing the name part of the email address. Stored as scrypt hashes (N = 2^15, r = 8, p = 3), never as the password. |
| Second factor | Every account must also give a six digit code from an authenticator app (TOTP). A code cannot be used twice. The authenticator secret is sealed with `AUTH_SECRET`. |
| Lockout | After `auth.max_failed_logins` (5) failures, the account locks for `auth.lock_minutes` (15). Each failure message is the same, so it never reveals which part was wrong. |
| Sessions | Random tokens, stored only as hashes, in a cookie that page scripts cannot read, sent only over HTTPS on a hospital installation, and not with forms posted from other sites. A session ends after `auth.session_hours` (12) or `auth.idle_minutes` (60) of no activity. |
| Rate limits | `limits.signin_per_minute` sign in attempts per address. |

## 12. How the platform protects itself

threat_model.md goes through each kind of attacker in turn. The main protections are:

| Protection | What it does |
|---|---|
| Signed collectors | Every record arriving is signed by a known, switched on collector, inside a five minute window. Secrets are worked out, never stored. |
| Least privilege | Only collectors with the respond permission can read or acknowledge isolation requests. A Windows group's secret can only send Windows events. |
| Human approval | No machine is ever isolated without a signed in person's approval, and never on one log source alone. |
| HTTPS everywhere | Caddy serves the console over HTTPS with the installation's own certificate authority; plain HTTP is sent to HTTPS. |
| Pinned trust | Windows collectors trust only the installation's own authority. |

| Protection | What it does |
|---|---|
| Content Security Policy | Pages may only run scripts carrying a fresh one time number (nonce), so injected script cannot run. Pages cannot be framed by other sites. |
| Cross site protection | Forms only accept posts from the installation's own pages; the backup endpoints check the `Origin` header. |
| Sealed secrets | Records database passwords, FHIR secrets and keys, and authenticator secrets are encrypted with AES 256 GCM before they are stored. |
| Tamper evident audit log | Each entry carries a SHA256 hash of itself and the entry before it. Changing or removing any past entry breaks the chain, and `audit_verify()` reports where. The database refuses updates and deletes on the log. |
| Locked backups | Backups made in the console are encrypted with AES 256 GCM under a key made from the administrator's passphrase by scrypt. A changed file is refused. |
| Minimal appliance | No SSH or other remote sign in. The maintenance account signs in only at the computer's own keyboard. |
| No secrets in the code | Every secret is made at installation or set in the host, never written in the code. CodeQL scans the code and ZAP attacks a running installation on every change. |

Security problems should be reported privately, as SECURITY.md describes.

## 13. Backups, restores and updates

**A backup holds everything**: the database (incidents, sources, settings, accounts, connections, audit log), the secrets, and the installation's certificate authority. Restored onto a new installation, every account, connection and Windows computer keeps working.

**Making one.** An administrator opens **Settings**, **Backups**, chooses a passphrase of at least `backups.passphrase_min_length` characters and presses **Make the backup**, then **Download the backup**. The file waits on the installation for `backups.keep_hours` (24), then is removed.

**The file.** A `.htmbackup` file starts with `HTMBACKUP`, then holds the encryption settings and the contents encrypted with AES 256 GCM. The key is made from the passphrase with scrypt (N = 2^17, r = 8, p = 1). The passphrase is never stored: a lost passphrase cannot be recovered.

**Restoring one.** From **Settings**, **Backups**, or, on a new installation nobody has set up, from the first visit page with the setup code. The administrator chooses the file, types its passphrase and types `RESTORE`. The backup service then:

1. decrypts and checks the file; a wrong passphrase or a changed file is refused with nothing changed;
2. refuses a backup that holds database changes this version does not know, as one made by a newer version does;
3. replaces the database with the backup's, in one transaction, then applies any database changes the backup's version did not have;
4. ends every session, so everyone signs in again with the restored accounts;

5. restores the secrets, keeping the new installation's own database password;
6. restores the certificate authority, so a new certificate is made from it;
7. writes the restore to the audit log.

The console, background service and HTTPS server notice their keys or authority changed and start again by themselves within seconds.

**Updating the appliance** to a new version: make a backup on the old one, install the new appliance, restore the backup on its first visit page, then give the new one the old one's address. **Updating a Docker installation**: `git pull` and `docker compose up -d --build`; data and secrets are kept.

## 14. Running it day to day

### Health

| Check | How |
|---|---|
| Is it up? | `GET /api/health`, or the appliance's screen, which says **Running.** |
| Is detection running? | The console overview shows "Detection service running", from the background service's heartbeat. |
| Are logs arriving? | **Sources** shows each source, its last event, its learning period and silent sources (none for `console.source_silent_after_minutes`). |
| Is the log receiver running? | The Sources page warns when the receiver has not reported in for a minute. |
| Is the audit log intact? | **Audit**, or `select * from audit_verify()` |

### Logs of the platform itself

On a Docker installation, `docker compose -f docker-compose.hospital.yml logs SERVICE`, for example `logs worker` or `logs backup`. On the appliance, the same after the maintenance sign in, in `/opt/htm`. While the appliance is starting, it also writes each part's state to its serial port every 10 minutes, so a log can be taken from an installation that never finishes starting.

### Common problems

| Symptom | Likely cause and what to do |
|---|---|
| Browser warns about the certificate | Expected until hospital computers trust the installation's authority. Download it from **Sources**, **Windows computers**. |
| Browser says it cannot connect | The platform is not running yet. Wait for **Running.** on the screen, and type the address with `https://`. |
| A device's logs do not appear | Accept the device under **Sources**. Check it sends to port 514 of the installation's address. |
| A Windows group stops sending | The group is switched off, the computer cannot reach port 443, or the installation's address changed. Its `collector.log` says which. |

| Symptom | Likely cause and what to do |
|---|---|
| The address changed | Set a fixed address with N on the appliance's screen, or ask for a DHCP reservation. Windows computers set up at the old address must be set up again. |
| Backups page says the service is not running | Restart the installation. |

## 15. How it is tested

Every change runs these checks on GitHub ( `.github/workflows/ci.yml` ), and each one fails the change if what it proves stops being true.

| Check | What it proves |
|---|---|
| Console tests | Record formats, signing, passwords, codes, image checks, backup files. With coverage reporting and a floor. |
| Engine tests | Simulator, rules, models, receiver formats, FHIR, forwarding, connector. With coverage reporting and a floor. |
| Database and end to end | Every migration applies twice; visitor roles can read nothing; the audit log refuses and detects tampering; a short evaluation that fails if any attack is missed; the audit chain intact afterwards; detection checks on a fresh database. |
| One command demonstration | Starts from nothing and catches an attack. |
| Hospital installation | Starts from nothing over HTTPS and checks it is locked down; then a backup made with clicks, the installation removed, a new one started and the backup restored onto it in a real browser. |
| Dynamic analysis | ZAP scans and attacks a running installation, as a visitor and signed in, and fails on anything of medium risk or higher not accepted with a proven reason. |
| Windows collector | Installs the collector on a Windows computer as an administrator would, makes events, runs the scheduled task, and checks what a local stand in receiver, which checks every signature, receives. Also checks a wrong secret is refused and explained, and that uninstalling works. |

CodeQL scans the JavaScript, TypeScript and Python code and the workflows on every change and every week, with its extended security queries ( `.github/workflows/codeql.yml` ). | Appliance scripts | Shell scripts checked with ShellCheck, the virtual machine description checked. |

For each release, `.github/workflows/release.yml` builds the appliance, starts it in a virtual machine, switches it off part way through its first start, starts it again, sets it up in a browser, sets and removes a fixed address, makes a backup, and installs from the USB stick with Secure Boot on, before the downloads are attached.

Measured coverage at 0.9.0, reported honestly: the console's library code 26% of lines, the engine 52% from unit tests alone (more when the end to end run is included). The floors only go up.

## 16. Standards and outside services used

| Standard or service | Used for |
|---|---|
| HL7 FHIR R4, AuditEvent | Reading patient record access from record systems |
| SMART Backend Services, OAuth 2.0 | Signing in to FHIR servers without a person |
| Syslog, RFC 3164 and RFC 5424 | Receiving router, firewall and server logs; sending CEF |
| CEF (Common Event Format) | Forwarding incidents to traditional SIEMs |
| Windows Event Log | What the Windows collector reads |
| MITRE ATT&CK | Naming the attacker technique behind each rule |
| HMAC SHA256 | Signing every request between parts |
| AES 256 GCM, scrypt, HKDF | Sealing secrets, hashing passwords, locking backups |
| TOTP (RFC 6238) | Authenticator app codes |
| HIPAA Security Rule, NIST CSF 2.0, HHS 405(d) | The compliance mapping on the Compliance page |
| PostgreSQL, MySQL, MariaDB | Records databases the platform can read |
| Resend | Sending invitation emails (optional) |
| Supabase and Vercel | Hosting the public demonstration (optional) |
| scikit-learn | The Isolation Forest models |
| Next.js, React, PostgreSQL, Caddy, Docker, Ubuntu | What the platform is built on |
| GitHub Actions, CodeQL, Dependabot, ZAP | Testing, code scanning, dependency updates, attack scanning |
| CICIoMT2024 dataset, Synthea | Optional research data for the demonstration |

## 17. Glossary

| Word | Meaning |
|---|---|
| Alert | One detector finding one thing worth a look. |
| Appliance | A ready made computer system, here a virtual machine or USB installer, that runs the platform with no setup beyond clicks. |
| ATT&CK | MITRE's public catalogue of the techniques attackers use. Each rule names one. |
| Audit log | The platform's own record of everything that happened, chained so changes show. |
| Band | Low, medium or high: what an incident's confidence means for what happens next. |
| CEF | Common Event Format, a text layout many security systems read. |
| Certificate authority | The installation's own issuer of HTTPS certificates. Computers that trust it see no warning. |

| Word | Meaning |
|---|---|
| Collector | Anything that sends logs to the platform with a signature: the Windows collector, the receiver, the background service. |
| Confidence | How likely an alert or incident is to be a real problem, between 0 and 1. |
| Connector | A small program that carries out approved isolations in an endpoint product. |
| DHCP | How a network gives a computer its address automatically. |
| Endpoint product | Security software on computers that, among other things, can cut them off the network. |
| Event | One log record turned into the platform's common format. |
| FHIR | The international standard for exchanging health data. |
| HMAC | A way of signing a message with a shared secret, so the receiver can prove who sent it and that it was not changed. |
| Incident | Related alerts grouped into one problem for a person to review. |
| Isolation | Cutting one computer off the network so an attack cannot spread from it. |
| Isolation Forest | A learning method that spots unusual activity without being shown attacks. |
| Learning period | The first days after a kind of log arrives, while the platform learns what is normal. |
| Migration | A file that creates or changes the database's tables. |
| Nonce | A number used once. Here, a fresh one on every page decides which scripts may run. |
| SIEM | A central system where large organisations collect security alerts from many tools. |
| Syslog | The common way routers, firewalls and servers send their logs over the network. |
| TOTP | Time based one time passwords: the six digit codes an authenticator app shows. |

## 18. Appendix: every setting

These are the settings as shipped with a hospital installation of 0.9.0. Administrators change them under **Settings**; changes take effect without a restart. On the demonstration, `engine.simulate` and `response.simulated_connector` are on.

### Accounts and sign in

| Setting | As shipped | What it does |
|---|---|---|
| `auth.idle_minutes` | 60 | A session ends after this long without activity. |
| `auth.lock_minutes` | 15 | How long a locked account stays locked. |
| `auth.max_failed_logins` | 5 | Failed sign ins before an account is locked. |
| `auth.password_min_length` | 12 | Shortest password accepted. |
| `auth.session_hours` | 12 | A session ends this long after sign in. |
| `auth.setup_link_hours` | 48 | How long an invitation link works. |

### Backups

| Setting | As shipped | What it does |
|---|---|---|
| `backups.follow_minutes` | 180 | Minutes the backup page waits for a backup or restore to finish before saying it took too long. |
| `backups.keep_hours` | 24 | Hours a finished backup waits on the installation to be downloaded before it is removed. Uploaded backups that were never restored are removed after the same time. |
| `backups.max_upload_mb` | 20480 | Largest backup file, in MB, that can be uploaded to restore. |
| `backups.passphrase_min_length` | 12 | Shortest passphrase a backup can be locked with. The passphrase is never stored, so a lost one cannot be recovered. |
| `backups.restore_confirm_word` | RESTORE | The word to type to confirm a restore, which replaces everything on this installation. |

### Console display

| Setting | As shipped | What it does |
|---|---|---|
| `console.assets_page_limit` | 500 | Most assets listed at once. |
| `console.audit_page_limit` | 200 | Most audit entries listed at once. |
| `console.incidents_page_limit` | 100 | Most incidents listed at once. |
| `console.overview_window_minutes` | 60 | Length of the window the overview and asset counts cover. |
| `console.recent_events_limit` | 15 | Rows in the latest events table. |
| `console.refresh_seconds` | 15 | How often the console asks for fresh data. Zero turns refresh off. |
| `console.source_silent_after_minutes` | 10 | A source with no event for this long is shown as silent. |

### Public demo

| Setting | As shipped | What it does |
|---|---|---|
| `demo.auto_release_minutes` | 20 | While the demo is on, a machine stays isolated this long, then the background service releases it. Zero turns this off. |
| `demo.enabled` | false | Lets anyone use the public demo login. The demo visitor can launch scenarios and isolate or release machines, nothing else. |
| `demo.session_minutes` | 60 | A demo session ends this long after it starts. |

### Incident scoring

| Setting | As shipped | What it does |
|---|---|---|
| `detection.band_high` | 0.8 | Confidence at or above this makes an incident eligible for isolation. |
| `detection.band_medium` | 0.5 | Confidence at or above this puts an incident in the analyst queue. |
| `detection.correlation_window_minutes` | 45 | Alerts on the same machine or person within this many minutes join one incident. |
| `detection.high_min_sources` | 2 | Independent log sources that must agree before an incident can be high. |
| `detection.incident_max_hours` | 4 | An incident stops taking new alerts this long after it opened. |
| `detection.single_source_cap` | 0.79 | Highest confidence an incident can reach while too few sources agree. |

### Email

| Setting | As shipped | What it does |
|---|---|---|
| `email.from_address` | (empty) | Address invitations are sent from. Its domain must be verified with the email provider. Empty turns email off. |
| `email.from_name` | (empty) | Name shown as the sender. Empty uses the platform name. |

### Background service

| Setting | As shipped | What it does |
|---|---|---|
| `engine.bootstrap_hours` | 24 | Hours of history the service generates on a new installation before first training. |
| `engine.poll_seconds` | 2 | How often the service looks for new events and requests. |
| `engine.retrain_hours` | 6 | How often the models retrain. Zero turns scheduled retraining off. |
| `engine.simulate` | false | Whether the service runs the hospital simulator. |
| `engine.training_hours` | 24 | Hours of recent activity each training run learns from. |

### Learning from analysts

| Setting | As shipped | What it does |
|---|---|---|
| `feedback.global_after_entities` | 3 | A value dismissed as normal for this many entities becomes normal for all. |

| Setting | As shipped | What it does |
|---|---|---|
| `feedback.model_margin` | 1.25 | After a dismissal, a model alert for that entity must sit this multiple further from normal. |
| `feedback.normal_weight` | 50 | How heavily a window dismissed as normal counts in later training. |
| `feedback.threshold_factor` | 1.5 | After a dismissal, a threshold rule must see this multiple of the dismissed count for that entity. |

### FHIR connections

| Setting | As shipped | What it does |
|---|---|---|
| `fhir.max_pages` | 20 | Most pages read from a FHIR server in one go. |
| `fhir.page_size` | 100 | AuditEvent entries asked for in each request to a FHIR server. |

### Hospital

| Setting | As shipped | What it does |
|---|---|---|
| `hospital.name` | (empty) | The hospital's name, shown in the console. Set it in Setup. |
| `hospital.timezone` | UTC | The hospital's time zone. Times in its records with no zone of their own are read in this zone. |

### Ingestion

| Setting | As shipped | What it does |
|---|---|---|
| `ingest.max_batch_records` | 1000 | Most records accepted in one batch. |
| `ingest.max_body_bytes` | 4000000 | Largest request body the ingestion API reads. |
| `ingest.max_skew_seconds` | 300 | How far a request timestamp may differ from server time. |

### Learning period

| Setting | As shipped | What it does |
|---|---|---|
| `learning.days` | 7 | Days each kind of log is learned from, after its first record arrives, before unusual activity in it is reported. Clear signs of attack are reported straight away. Zero switches learning off. |

### Rate limits

| Setting | As shipped | What it does |
|---|---|---|
| `limits.api_per_minute` | 1200 | Requests one address may make to the public API routes each minute. |
| `limits.demo_per_minute` | 6 | Demo logins, scenario launches and isolation requests one address may make each minute as the demo visitor. |
| `limits.signin_per_minute` | 10 | Sign in attempts one address may make each minute. |

### Landing page pictures

| Setting | As shipped | What it does |
|---|---|---|
| `media.max_bytes` | 3000000 | Largest picture an administrator may upload, in bytes. Hosting limits a single upload to about 4 MB. |

### Anomaly models

| Setting | As shipped | What it does |
|---|---|---|
| `model.alert_quantile` | 0.995 | A window must score above this share of training windows before it can raise an alert. |
| `model.confidence_max` | 0.75 | Highest confidence a model alert can carry. |
| `model.confidence_min` | 0.35 | Confidence of a model alert that only just crosses its threshold. |
| `model.full_confidence_z` | 12 | Deviations from normal at which a model alert reaches its highest confidence. |
| `model.min_feature_z` | 3 | At least one feature must be this many deviations from normal. |
| `model.min_group_windows` | 200 | A peer group with fewer training windows than this is judged against everyone. |
| `model.min_training_windows` | 200 | A model with fewer training windows than this stays silent. |
| `model.sample_size` | 4096 | Training windows each tree samples. |
| `model.trees` | 200 | Trees in each Isolation Forest. |

### Platform

| Setting | As shipped | What it does |
|---|---|---|
| `platform.contact_email` | (empty) | Contact address shown on the landing page. Empty hides it. |

| Setting | As shipped | What it does |
|---|---|---|
| `platform.name` | Healthcare Threat Monitor | Working title shown in the console and the browser tab. |
| `platform.repository_url` | (empty) | Link to the public code repository, shown on the landing page. |

### Log receiver

| Setting | As shipped | What it does |
|---|---|---|
| `receiver.batch_seconds` | 5 | How often the receiver passes the logs it has collected on for storage. |
| `receiver.max_message_bytes` | 8192 | Longest single log message the receiver keeps. Longer messages are cut to this length. |
| `receiver.sender_per_minute` | 60000 | Most log messages kept from one device in a minute. More than this are counted and dropped. |

### Response

| Setting | As shipped | What it does |
|---|---|---|
| `response.connector_poll_seconds` | 3 | How often the simulated endpoint connector checks for isolation requests. |
| `response.simulated_connector` | false | Whether the background service acts as the endpoint. Switch off when a real connector is running. |

### Retention

| Setting | As shipped | What it does |
|---|---|---|
| `retention.events_hours` | 36 | Events older than this are removed. Alert evidence is kept with the alert. Keep it above the training hours. |

### Test attacks (demo)

| Setting | As shipped | What it does |
|---|---|---|
| `scenario.auto_interval_minutes` | 0 | Launch a scenario automatically this often. Zero turns it off. |
| `scenario.time_scale` | 0.25 | Speed of scenarios launched from the console. 1 is real time, 0.25 is four times faster. |

### SIEM forwarding

| Setting | As shipped | What it does |
|---|---|---|
| `siem.max_attempts` | 5 | Delivery attempts before a message is marked failed. |

| Setting | As shipped | What it does |
|---|---|---|
| `siem.syslog_host` | (empty) | Syslog destination for CEF messages. Empty turns syslog off. |
| `siem.syslog_port` | 514 | Syslog destination port. |
| `siem.syslog_protocol` | tcp | Syslog transport: tcp or udp. |
| `siem.webhook_url` | (empty) | Where incident messages are posted. Empty sends them to the built in receiver. |

### Records databases

| Setting | As shipped | What it does |
|---|---|---|
| `sources.batch_rows` | 500 | Most entries read from a records database in one go. |
| `sources.poll_seconds` | 30 | How often the background service reads new entries from each connected records database. |

### Uploads

| Setting | As shipped | What it does |
|---|---|---|
| `uploads.max_bytes` | 20000000 | Largest report or spreadsheet an administrator may upload, in bytes. Vercel refuses uploads over about 4.5 MB whatever this says. |

### Windows collectors

| Setting | As shipped | What it does |
|---|---|---|
| `windows.interval_minutes` | 1 | How often each Windows collector sends new events. Takes effect when a computer next downloads its settings. |